declaration. That way, you don’t have to check for null. You only need to check whether it is an empty collection, using the `Any` extension method.

5.2.5 Binding complex collections

The model binder also supports binding to collections of complex objects. When you work with collections of complex objects, you incorporate an index value to identify each element in the collection when you construct the form controls’ name attributes. The index can be one of two kinds:

- *Sequential index*—A numeric index that starts at zero and increases by an increment of one each time with no gaps
- *Explicit index*—An index made up of arbitrary but unique values of any type

THE SEQUENTIAL INDEX APPROACH

The following example illustrates the use of a sequential index. You will modify the Create page in the CountryManager folder for this. You will continue to use the `InputModel` class from the previous examples, but you will change the properties in the `PageModel` class to represent collections as well as alter assignment code in the `OnPost` handler accordingly.

Listing 5.16 The `CreateModel` class in the country manager

```
public class CreateModel : PageModel
{
    [BindProperty]
    public List<InputModel> Inputs { get; set; }
    public List<Country> Countries { get; set; } = new List<Country>();

    public void OnPost()
    {
```